

Business Model

1. What is being sold?

We are not selling a coding model.

We are not selling a prompt-rewriting utility.

We are selling **reduced supervision cost and increased trust in AI-assisted software engineering**.

The economic promise is:

Developers can delegate larger engineering tasks to coding agents while spending less time correcting, monitoring and verifying those agents.

The product creates value when it improves one or more of:

- accepted tasks per engineer;
- first-pass task acceptance;
- human interventions per accepted task;
- engineering time per accepted task;
- regression rate;
- agent token consumption;
- confidence in AI-generated changes;
- consistency across developers and agents.

The highest-value resource being saved is **human engineering attention**.

2. Initial customer

The first customer should not be a Fortune 500 enterprise.

Start with developers who already feel the problem intensely:

AI-native developers / technical founders / senior engineers

Typical characteristics:

- use Claude Code, Cursor, Codex or similar daily;
- delegate meaningful implementation work to agents;
- frequently run long agent sessions;
- already spend money on AI coding tools;

- experience context loss, repeated corrections and review fatigue;
- are comfortable installing CLI tooling;
- can immediately judge whether the product improves their workflow.

These users can provide dense feedback and measurable task data.

3. Initial wedge

The initial wedge is:

Make Claude Code require materially less babysitting on real repository tasks.

This is intentionally narrower than the ultimate company.

The first version can be distributed as:

- Claude Code plugin;
- local CLI/runtime;
- GitHub integration later;
- MCP interface for project memory/context;
- hooks for deterministic lifecycle control.

Claude Code is the distribution wedge.

The company itself should remain model-agnostic.

4. Expansion path

Phase 1 — Individual developers

Value:

- better task understanding;
- persistent project knowledge;
- automatic verification;
- fewer correction loops;
- lower token waste;
- task outcome reports.

Potential pricing hypothesis:

- free/local open-source tier;
- paid individual tier roughly in the developer-tool range.

Do not finalize pricing before measuring usage and willingness to pay.

Phase 2 — Small teams

Additional value:

- shared project memory;
- shared architecture decisions;
- team policies;
- team verification rules;
- cross-agent continuity;
- task analytics;
- standardized agent workflows;
- organization-level acceptance criteria.

Likely pricing model:

- per developer;
- usage-based verification;
- hybrid seat + usage.

Phase 3 — Business / enterprise

Additional value:

- central policy enforcement;
- model governance;
- audit trail;
- provenance;
- security controls;
- approval workflows;
- CI/CD integration;
- company-wide engineering memory;
- model routing;
- cost analytics;
- agent performance comparison;
- data residency/private deployment options.

Possible commercial model:

- annual contract;
 - seat minimum;
 - platform fee;
 - usage;
 - enterprise support.
-

5. Why customers pay

Do not anchor the value proposition on token savings alone.

Token savings are easy to understand but strategically weak because inference costs may decline.

The value hierarchy is:

1. **Correctness**
2. **Human attention saved**
3. **Engineering time saved**
4. **Fewer regressions**
5. **Token/API cost saved**

A senior engineer spending 25 minutes correcting an agent is usually more economically important than several dollars of inference.

The product should eventually quantify both.

Example report:

Last 30 days

AI tasks accepted:	84
Human interventions / accepted task:	2.1
Verification failures caught:	31
Estimated engineer time saved:	14.7h
Tokens avoided:	18.4M
Escaped regressions:	[measured]

All sample numbers are illustrative until measured.

6. Core business KPI

Human Interventions Per Accepted Task

This should be a major internal and external KPI.

Supporting metrics:

- first-pass acceptance rate;
- accepted task rate;
- median time to accepted result;
- cost per accepted task;

- tokens per accepted task;
 - auto-repair success rate;
 - verifier precision;
 - verifier false-positive rate;
 - escaped regression rate;
 - human clarification frequency.
-

7. Investor narrative

Do not pitch:

"We use a cheaper AI to improve Claude prompts."

That sounds like a feature.

Do not pitch:

"We give coding agents memory."

That sounds vulnerable to native platform improvements.

Do not pitch:

"We reduce tokens."

That sounds vulnerable to cheaper models.

Pitch:

We are building the reliability layer between developer intent and AI-written software.

Narrative:

1. Coding agents have made code generation dramatically cheaper.
 2. Developers are increasingly supervising agents instead of typing every line.
 3. The new bottleneck is specification, monitoring, correction and verification.
 4. Every major coding agent still requires humans to perform this supervisory loop.
 5. Our runtime automates that loop.
 6. We turn developer intent into a task contract, supply project knowledge, observe execution, independently verify results and automatically repair failures.
 7. Our key measure is **human interventions per accepted task**.
 8. Our long-term asset is a cross-agent dataset of real engineering intents, failures, repairs and accepted outcomes.
-

8. The platform thesis

The larger market transition is:

human-written software



human + AI-assisted software



human-supervised machine-written software



increasingly autonomous software engineering

As generation becomes commoditized, a new infrastructure layer is needed to own:

- what the human intended;
- what the organization knows;
- what the agent changed;
- whether the result is correct;
- why the result was accepted;
- what should be remembered next time.

That is the category we want to own.

9. Defensibility

Weak moats:

- RAG;
- embeddings;
- vector databases;
- prompt templates;
- model wrappers;
- one-vendor hooks;
- a single MCP server.

Potential strong moat:

1. Cross-agent system of record

Company knowledge and verification state should survive model changes.

2. Outcome dataset

Store:

task

- selected context
- execution trace
- failure
- repair
- accepted result

This becomes a proprietary corpus of how agents succeed and fail on real software.

3. Project-specific learned invariants

Example:

Authentication requires a verified email.

Store with:

- evidence;
- source;
- scope;
- freshness;
- confidence;
- relevant symbols.

Retrieve only when relevant.

4. Verification intelligence

Over time learn:

- what needs testing for each task class;
- what failures existing tests miss;
- what files are commonly forgotten;
- what architecture constraints are violated;
- which agents perform best for which work.

5. Team workflow lock-in

If the product becomes where a team stores:

- engineering decisions;
- verification rules;
- task history;
- agent performance;
- accepted outcomes;

switching costs become meaningful.

10. Competitive strategy

The company should **benefit when base models improve**.

If Claude or Codex becomes better:

- implementation succeeds more often;
- repair loops shrink;
- verification costs decline;
- our reliability runtime delivers more value.

The company should not need to beat model vendors at model intelligence.

It should orchestrate them.

11. Business risks

Risk: Vendor absorption

Anthropic/OpenAI/Cursor/GitHub add native planning, memory and verification.

Mitigation:

- remain cross-agent;
- own team state;
- own outcome history;
- integrate multiple vendors;
- become system of record rather than a single-agent enhancement.

Risk: Models become reliable enough that supervision disappears

Mitigation:

- enterprise verification, governance, auditability and evidence will still matter;
- shift from personal supervision to organizational policy enforcement;
- runtime can become infrastructure for autonomous agents rather than a correction tool.

Risk: Verification becomes too expensive

Mitigation:

- deterministic checks first;
- cheap models for classification/triage;
- expensive verification only when needed;
- reuse cached project knowledge;

- learn task-specific verification strategies.

Risk: Developers dislike another layer

Mitigation:

- preserve existing workflow;
- install once;
- minimal UI;
- prove measurable improvement;
- avoid forcing a new IDE.

Risk: Privacy/source-code concerns

Mitigation:

- local-first architecture where possible;
 - open-source client/integration;
 - explicit data controls;
 - cloud optional at first;
 - enterprise private deployment later.
-

12. Pricing hypotheses

These are hypotheses, not commitments.

Individual

Possible:

- free local tier;
- \$20–\$50/month paid tier;
- usage included up to limits.

Team

Possible:

- \$30–\$80/user/month depending on verification/analytics;
- shared memory and policies;
- optional usage charge.

Business

Possible:

- \$10k–\$50k/year for smaller organizations;
- higher depending on seats, usage and integrations.

Enterprise

Potential:

- \$100k+ annual contracts when governance, security, private deployment and compliance become central.

Do not optimize pricing before product validation.

First prove:

"This measurably reduces human supervision on real software tasks."

13. Landing-page core message

Primary:

Intent → Verified Result

Supporting:

Your coding agent can write the code.

We make sure it understands the job, finishes the job, and proves that it did.

Problem visualization:

Today:

prompt
prompt
correction
prompt
explanation
correction
test
fix
prompt
fix

Future:

intent

↓
verified result

Metric visualization once real measurements exist:

Human interventions / accepted task

Vanilla agent [REDACTED]
With [Product] [REDACTED]

Never use invented values as real metrics.

14. Business-model source of truth

The business is not "sell better prompts."

The business is:

Sell reliable delegation of software-engineering work to AI agents.

The initial buyer is the power user.

The long-term buyer is the engineering organization.

The initial feature is reduced babysitting.

The long-term platform is the control plane and system of record for autonomous software engineering.